# Barclays Hackathon

**Problem statement:**
Pre-Delinquency Intervention Engine

**College**: College of Engineering Guindy, Chennai

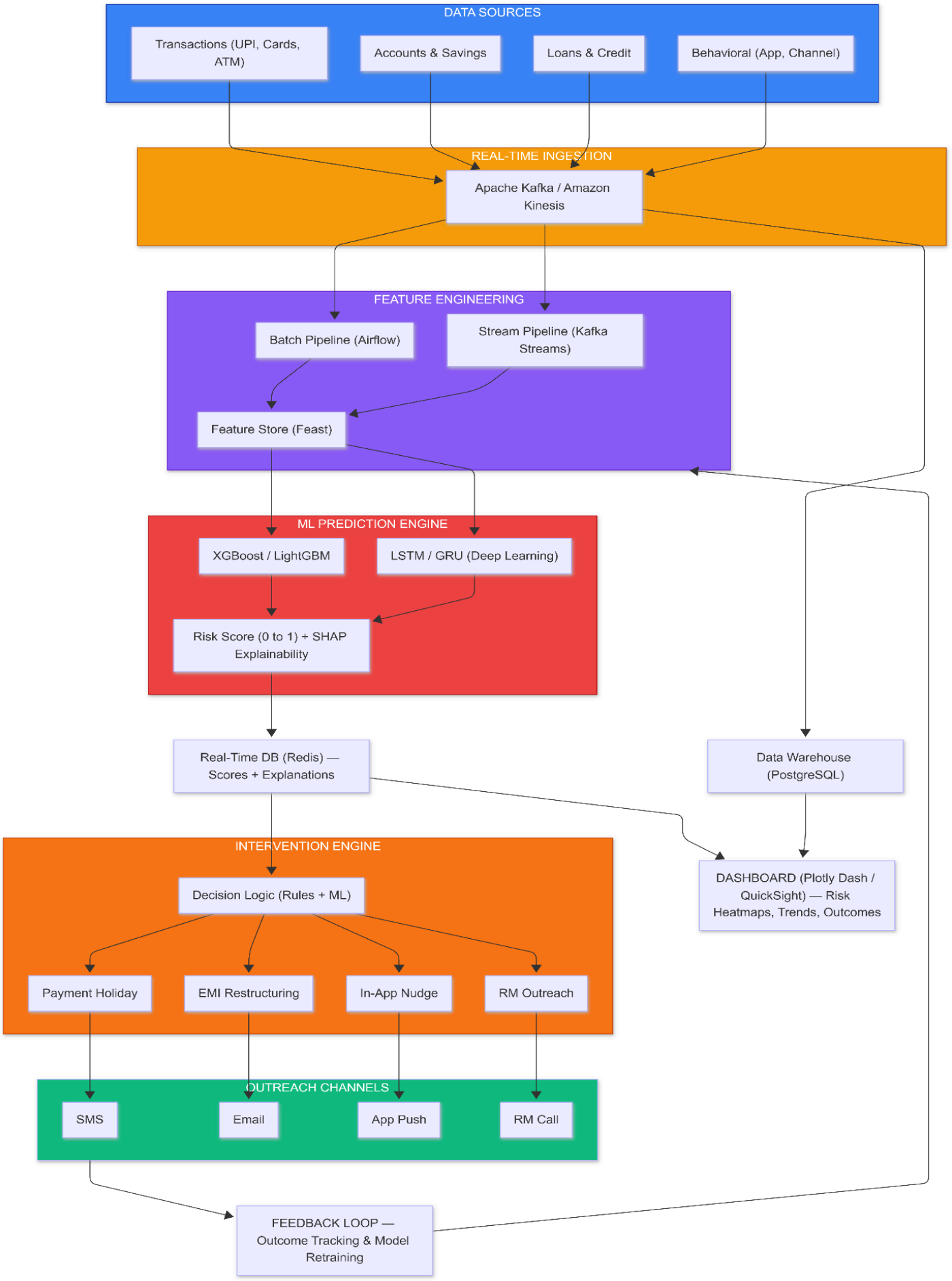
## Abstract

Banks traditionally manage credit risk reactively intervening only after a missed payment, when recovery likelihood is low and customer trust is already damaged. Early stress indicators like spending shifts, savings drawdowns, and delayed credits remain scattered across siloed systems, invisible until it's too late.

Our Pre-Delinquency Intervention Engine addresses this by analyzing real-time transaction streams and historical behavioral patterns to predict payment default 2–4 weeks in advance. Built on an event-driven architecture using Apache Kafka, Flink, and Feast, the platform ingests diverse data sources UPI, ATM, account balances, bill payments and computes real-time risk scores through an ensemble ML model with SHAP explainability. When risk thresholds are breached, an alerting engine routes tailored interventions (payment holidays, EMI restructuring) through each customer's preferred channel. A closed-loop feedback system captures outcomes to enable continuous model retraining.

By shifting to proactive, empathetic engagement, the engine reduces credit losses, lowers collection costs, and preserves customer relationships.

# System Architecture



# Proposed System

## Date Ingestion Layer:

- Real-time transaction and account updates from systems such as UPI, ATM networks, and core banking platforms are captured using Debezium-based Change Data Capture (CDC), which streams database changes directly from transaction logs.

- The captured events are ingested into Apache Kafka, serving as the central streaming backbone responsible for buffering records, managing throughput, and handling volume spikes.

- Kafka topics are partitioned using a stable key , ensuring deterministic processing and preserving the correct chronological order of events.

- Incoming events can then be processed using a stream processing framework (Apache Flink), where they undergo validation, schema enforcement, normalization, and contextual enrichment.

- The refined events are delivered to downstream analytics and feature systems.

## Feature Engineering and storage:

**1. Stream Processing with Apache Flink**

Flink continuously consumes events from Kafka and performs two critical tasks:

**a) Enrichment**

- -Each transaction is enriched with a **merchant risk score** by looking up the merchant's MCC or ID in a pre-loaded enrichment table (e.g., stored in Redis or broadcast as a static dataset). This score reflects the riskiness of the merchant category (e.g., payday lenders = 0.9, grocery = 0.1).
- -The enriched transaction now contains fields like `merchant_risk_score` and `risk_category`.

**b) Rolling Feature Computation**

Flink maintains state for each customer and computes real-time features over sliding windows (e.g., last 7 days, 30 days). Examples:

- discretionary_spend_7d – total spend in dining/entertainment.
- atm_withdrawals_count_7d – number of ATM withdrawals.
- lending_app_txn_count_7d – count of transactions to known lending apps.
- weighted_lending_risk_7d – sum of merchant_risk_score for lending-app transactions.
- savings_balance_pct_change_7d – percentage change in balance (from account_updates stream).
- failed_autodebits_count_7d – number of failed auto-debit attempts.

These features are updated incrementally as new events arrive. After each update, Flink writes the new feature values to the "Feast online store" (Redis) via Feast's Python SDK or a dedicated sink. This ensures that the latest values are always available for low-latency retrieval.

**2. Batch Processing with Apache Spark (Orchestrated by Airflow)**

Daily (or hourly) batch jobs complement the real-time features with historical baselines and customer-specific attributes that do not change rapidly.

**a) Baseline Feature Computation**

- Salary delay: For each customer, analyse historical salary credits to determine the expected day of the month. Compute `salary_delay_days` = current_day – expected_day if salary not yet received.

- Utility payment delay: Average days past due for utility bills over the last 3–6 months.
- Discretionary spend trend: Compare last 7 days' discretionary spend to the same period last month.
- Demographic and product data: Age, tenure, product holdings, credit bureau scores.

**b) Fairness & Bias Checks**

During batch, we also compute demographic group membership (e.g., age bracket, location) to later monitor model fairness. This data is stored alongside features but not used in the model directly – it's for auditing.

**c) Writing to Feast**

All batch-computed features are written to Feast's offline store (e.g., PostgreSQL). Feast then runs a materialization job (triggered by Airflow) that transfers these features to the online store (Redis), making them available for real-time serving. Materialization can be incremental or full refresh.

**3. Feature Store – Feast as the Central Hub**

Feast acts as the unified feature registry and storage layer. It provides:

- Feature definitions: All features (streaming and batch) are defined in Feast with names, types, and sources.
- Online store (Redis): Holds the latest feature values for every customer, updated continuously by Flink and periodically by batch materialization.
- Offline store (PostgreSQL): Stores historical feature values with timestamps, used for training data export and point-in-time joins.
- Feast guarantees that the same feature definitions are used for both training and serving, eliminating training-serving skew.

**4. On-Demand Feature Vector Assembly**

When a risk score is needed – either triggered by a new transaction, a scheduled job, or an agent's request – the Scoring Service (a FastAPI microservice) performs the following steps:

**1. Receive request** with “customer_id” (e.g., “Alex”).

**2. Call Feast online retrieval:**

```
        feature_vector = store.get_online_features(
        feature_refs=[
        "transaction_features:discretionary_spend_7d",
        "transaction_features:atm_withdrawals_count_7d",
        # ... other features
        ],
    entity_rows=[{"customer_id": "Alex"}]
  ).to_dict()
```

Feast retrieves the requested features directly from Redis in sub-millisecond time. The result is a dictionary mapping feature names to current values.

**3. Assemble feature vector:** The dictionary is converted into a fixed-order array (or directly passed as a DataFrame row) matching the input format expected by the ML model.

**4. Model inference:** The feature vector is fed into the deployed model (XGBoost), which outputs a probability of delinquency (e.g., 0.72).

**5. Post-processing:** The probability is mapped to a credit score (e.g., 850 – probability * 550) and stored in both Redis (for fast access) and PostgreSQL (for historical tracking).

**6. Optional GenAI integration:** For dashboard display or automated outreach, the top SHAP values from the model (computed alongside the prediction) can be sent to a GenAI service to generate a natural-language explanation.

**7. Feedback Loop and Continuous Improvement:**

To ensure the model stays accurate and fair, a feedback mechanism is integrated:

- After each intervention, the CRM publishes a “feedback_event" to Kafka (e.g., “customer_id”, “intervention_type”).
- A dedicated consumer updates the labels table in Feast’s offline store.
- Periodically, an Airflow DAG triggers model retraining:
- It queries Feast’s offline store for historical features with the latest labels (using point-in-time correctness).
- It trains candidate models, applies fairness checks (e.g., with AIF360), and compares them with the current champion.
- If a new model is better, it is registered in MLflow and deployed to the scoring service.

## Predictive ML Model:

- To capture both the *current state* and the *temporal progression* of financial stress, the ML layer uses a hybrid modeling approach.

- First, an XGBoost classifier is trained on tabular behavioral features to estimate the immediate risk of delinquency based on the customer’s present financial condition.

- This model is highly effective for noisy financial data, supports nonlinear feature interactions, and provides strong interpretability, which is essential for regulatory and governance requirements.

- Second, a Long Short-Term Memory (LSTM) neural network is trained on sequences of customer feature vectors over time (for example, a 30-day sliding window).

- The LSTM learns patterns of gradual deterioration-such as steady balance decline, repeated small delays, or rising cash dependence-that are often invisible in single-day snapshots. By modeling behavioral trajectories, the LSTM detects emerging distress trends earlier than traditional models.

## Risk Scoring and Storage:

- The outputs of the two models are combined through a weighted ensemble to produce a single, stable delinquency risk score. This fusion reduces false positives, improves sensitivity to early warning signals, and ensures consistent performance across customer segments.

- The resulting probability score is mapped into risk tiers (normal, emerging stress, high risk) and updated in near real time as new data arrives.

- An explainability layer based on SHAP (SHapley Additive exPlanations) accompanies every prediction. It identifies the most influential behavioral drivers-such as delayed salary, savings drawdown, or utility payment shifts-ensuring transparency, fairness, and audit readiness.

- Continuous model monitoring, retraining, and bias evaluation further ensure that the ML layer remains accurate, stable, and compliant over time.

## Alerting and Intervention Engine:

- **Risk Tiers** - Scores trigger alerts only on tier transitions (Critical >0.7 / Watch >0.5 / Stable), avoiding noise from minor fluctuations

- **Signal-Aware Routing** - SHAP explanations determine the action: salary delay → payment holiday, savings drop → EMI restructuring, lending app spike → wellness check-in.

- **Channel Optimization** - Outreach goes through each customer's highest-engagement channel (SMS, email, app, or RM call) based on historical response data.

- **Cooldown & Escalation** - 7-day cooldown prevents over-contacting; if risk worsens with no response, intensity escalates (nudge → SMS → RM call).

- **Closed-Loop Feedback** - Every intervention outcome (paid, restructured, defaulted) is tracked and fed back into model retraining to improve future decisions.

## Visualization and Dashboard:

- **Portfolio Risk Heatmap -** A single-view breakdown showing customer distribution across risk tiers (Critical / Watch / Stable), filterable by product, region, and segment for quick portfolio-level decisions.

- **Trending Customers Panel** - A ranked list of customers with the fastest-deteriorating risk scores week-over-week, with top stress signals (e.g., "salary delayed 9 days, savings down 18%") surfaced inline for immediate action.

- **Intervention Tracker** - Logs every outreach - type, channel, timestamp - alongside response rates and outcomes (payment made, restructured, defaulted), measuring what interventions are actually working.

- **Customer Deep-Dive View** - Click any customer to see a timeline of their financial behavior: income patterns, spending shifts, balance trends, and risk score history - giving RMs full context before outreach.

- **Model Health Monitor** - Displays live prediction metrics (precision, recall, AUC) with automated drift detection alerts, flagging when model accuracy degrades and retraining is needed.

## Feedback Loop and Model Retraining:

- **Outcome Labeling -** Every intervention is tracked to its final result (customer paid, restructured, or defaulted), creating a continuously growing labeled dataset that reflects real-world intervention effectiveness.

- **Drift Detection** - Statistical monitors compare current prediction distributions against baseline; when accuracy degrades beyond a threshold (e.g., AUC drops >5%), an automated retraining trigger fires.

- **Scheduled Retraining** - Airflow pipelines retrain models on a fixed cadence (e.g., bi-weekly) using the latest outcome-labeled data, ensuring the model adapts to evolving customer behavior and macroeconomic shifts.

- **Champion–Challenger Testing** - New models are deployed alongside the existing one in shadow mode; predictions are compared, and the new model only replaces the current one if it outperforms on key metrics (precision, recall, fairness).

- **Bias & Fairness Audits** - Each retrained model is automatically evaluated for demographic fairness (across income groups, regions, gender) before promotion, ensuring interventions remain equitable and regulatory compliant.

# Tech Stack

- **Data Ingestion -**Apache Kafka, Debezium

- **Stream Processing -**Apache Flink
- **Batch Processing -**Apache Airflow, Apache Spark
- **Feature Store -**Feast, Redis, PostgreSQL
- **Data Warehouse -**ClickHouse / PostgreSQL
- **ML Training -**XGBoost, LightGBM, PyTorch, scikit-learn
- **Explainability -**SHAP, LIME
- **Model Serving -**BentoML, MLflow
- **Risk Score Storage -**Redis / Apache Cassandra
- **Alerting & Intervention -**Python Rules Engine, Celery
- **Outreach Channels -**Apprise
- **Visualization -**Plotly Dash, Apache Superset, Grafana
- **Feedback & Retraining -**Apache Airflow, MLflow, Evidently AI
- **Bias & Fairness -**Fairlearn, AIF360
- **Infrastructure -**Docker, Kubernetes, GitHub Actions

# Future Scope

**1.GenAI-powered outreach** - Personalized, empathetic messaging and chatbot-based financial counseling

**2.Open banking integration** - Cross-institutional data for a holistic customer view

**3.Graph neural networks** - Detect household-level stress across linked accounts

**4.Alternative data** - Telecom, rent, and social signals for thin-file customers

**5.Multi-product expansion** - Mortgages, insurance, BNPL coverage

**6.Regional risk modeling** - Adapt to local economic conditions

**7.Reinforcement learning** - Continuously optimize intervention timing, channel, and offer type